Graph Theory

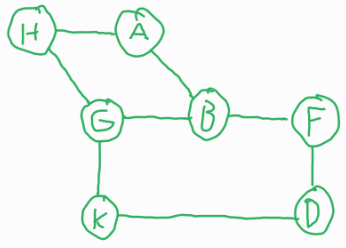
Definitions:

- A graph G is a pair of sets $V(G), E(G)$, where $V(G)$ is a non-empty set of vertices and $E(G)$ a set of pairs of vertices $\{u, v\}$, called edges

Example: $V = \{A, B, D, F, G, H, K\}$

$E = \{AB, DF, BG, GK, FB, AH, DK, GH\}$

$G =$



- An independent set is a subset of vertices where no edges exist between members of the set
  - in the above graph, $\{K, H, B\}$ is an independent set.
- A dominating set is a subset of vertices such that all vertices in the graph are either in the set or neighbouring a member of the set.
  - in the example graph, $\{A, K, F\}$ is a dominating set.
- The neighbourhood of a vertex is the set of neighbours of that vertex
  - in the example, the neighbourhood of B is $\{A, F, G\}$
- The degree of a vertex, $deg(v)$, is the number of neighbours of that vertex.
  - $\delta(G)$ denotes the smallest degree in the graph
  - $\Delta(G)$ denotes the largest degree in the graph
  - $deg(A) = 2$, $\delta(G) = 2$, $\Delta(G) = 3$ with G = example graph

Handshaking Lemma: Let $G = (V, E)$ be a graph. Then $\sum_{v \in V} deg(v) = 2|E|$

=> in an undirected graph, the number of vertices with an odd degree is even.

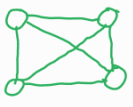
Graph classes:

- P_n

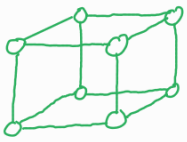
- C_n

- $K_{p,q}$
  - a graph that can be partitioned into two vertex sets containing p and q vertices, and all possible edges between the sets and none between members of the same set.
  - => has $p \times q$ edges.

- k_n
  - all possible edges on n vertices



- Q_n
  - n-dim hypercube

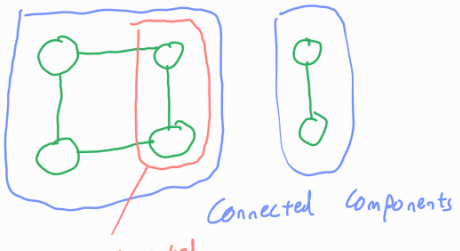


Paths and Cycles:

- A walk in a graph is a sequence of adjacent nodes through a graph
- A path is a walk such that all nodes in the walk are unique.
- A circuit is a walk that starts and ends at the same node
- A cycle is a path that starts and ends at the same node
- the length of a path is the number of edges in it.
- The distance between two vertices is the length of the shortest path between them, or infinity if no such path exists.
- The diameter of a graph is the maximum distance between two vertices in it.
- A Eulerian Cycle is a cycle that starts and ends at the same node, and traverses every edge of the graph
  - A graph has such a cycle iff each of its vertices has even degree
- A Hamiltonian Cycle is a cycle that starts and ends at the same node, and traverses every node of the graph

Connectivity:

- A graph, G is connected if for every pair of vertices in G there exists a path between them,
- A connected component of G is a maximal connected subgraph of G. If G is a connected graph it just has one connected component (the whole graph)



  - Every graph $G=(V,E)$ has at least $|V|-|E|$ connected components
  - If $G=(V,E)$ is connected, then $|E| \geq |V|-1$
- A directed graph is weakly connected if the undirected graph obtained by forgetting directions is connected
  is strongly connected if for every pair of vertices there exists a path between them in both directions.

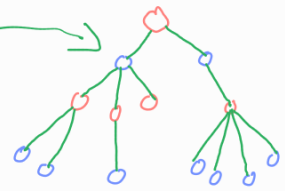
Isomorphism:

- Two graphs G and G' are isomorphic if there is a renaming of the vertices of G that coincides with G'.
  ↓
  i.e. $G \cong G'$
  - If $G \cong G'$, then they share all their characteristics, e.g cycles, degree sequence, independent sets etc., but these can only be used to prove $G \ncong G'$.

Trees and Forests:

- A forest is a graph with no cycles.
- A tree is a connected forest.
- A rooted tree is a tree with one vertex designated as the root.

- Every connected graph $G(V, E)$ has a spanning tree $T(V, E')$
  - This spanning tree is G if G has no cycles, otherwise remove an edge from each cycle in G to produce a tree T consisting of all vertices V of G.
- Every tree with at least 2 vertices contains a leaf
- A connected graph on n vertices is a tree iff it has $n-1$ edges.
- There is a single, unique path between any two distinct vertices in a tree.
- Every tree is bipartite

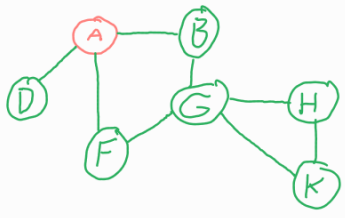


Breadth-First Search

```
BFS (G, s)
1  for each vertex u ∈ V[G] − {s} do
2      colour[u] ← WHITE        white = undiscovered
3          d[u] ← ∞
4              π[u] ← NIL       -> array to record predecessors
5  colour[s] ← GREY             Grey = discovered, unprocessed
6  d[s] ← 0
7  π[s] ← NIL
8  Q ← ∅
9  ENQUEUE(Q, s)
10 while Q ≠ ∅ do
11     u ← DEQUEUE(Q)
12         for each v ∈ Adj[u] do
13             if colour[v] = WHITE
14                 then colour[v] = GREY
15                     d[v] ← d[u] + 1     - for unweighted graphs.
16                     π[v] ← u
17                     ENQUEUE(Q, v)
18         colour[u] ← BLACK        Black = discovered, processed
```

Example: Source = A

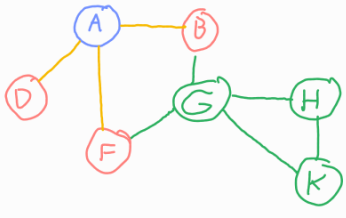
- white
- grey
- black



Q = A

	A	B	D	F	G	H	K
Colour	●	●	●	●	●	●	●
distance, d	0	∞	∞	∞	∞	∞	∞
Predecessor, π	NULL	NULL	NULL	NULL	NULL	NULL	NULL

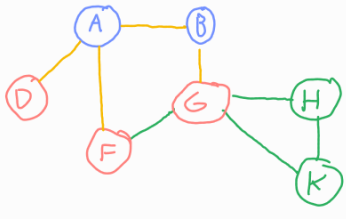
↓ Process A



Q = B, D, F

	A	B	D	F	G	H	K
Colour	●	●	●	●	●	●	●
distance, d	0	1	1	1	∞	∞	∞
Predecessor, π	NULL	A	A	A	NULL	NULL	NULL

↓ Process B



Q = D, F, G

	A	B	D	F	G	H	K
Colour	●	●	●	●	●	●	●
distance, d	0	1	1	1	2	∞	∞
Predecessor, π	NULL	A	A	A	B	NULL	NULL

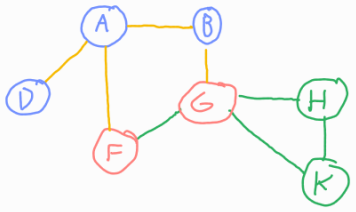
↓ Process D

Process:
- initialise source vertex as grey, all others white, enqueue source
- while the queue is not empty:
  - dequeue vertex v
  - set distance and predecessor for v
  - enqueue v's white neighbours
  - colour v black

Time Complexity:
- initialisation → $O(V)$
- queue operations → $O(V)$ as each vertex is enqueued and dequeued once.
- In the inner loop, the adjacency lists for a vertex is read once. This is executed once per vertex so the outer loop is $O(E)$ as the sum of the sizes of these lists

is E.

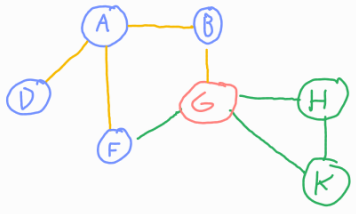
– $O(V+E)$ overall.



Q = F, G

	A	B	D	F	G	H	K
Colour	•	•	•	•	•	•	•
distance, d	0	1	1	1	2	∞	∞
Predecessor, π	NULL	A	A	A	B	NULL	NULL

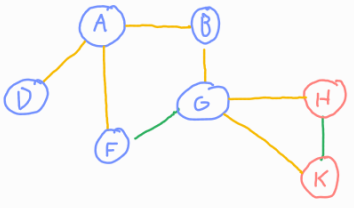
↓ Process F



Q = G

	A	B	D	F	G	H	K
Colour	•	•	•	•	•	•	•
distance, d	0	1	1	1	2	∞	∞
Predecessor, π	NULL	A	A	A	B	NULL	NULL

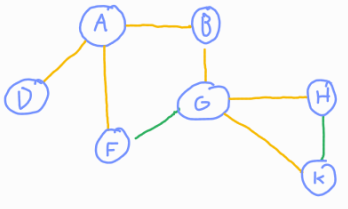
↓ Process G



Q = H, K

	A	B	D	F	G	H	K
Colour	•	•	•	•	•	•	•
distance, d	0	1	1	1	2	3	3
Predecessor, π	NULL	A	A	A	B	G	G

↓ Process H
Process K



Q = ∅

	A	B	D	F	G	H	K
Colour	•	•	•	•	•	•	•
distance, d	0	1	1	1	2	3	3
Predecessor, π	NULL	A	A	A	B	G	G

The highlighted edges of the processed graph form the breadth-first tree. These are the edges $(v, \pi[v])$ for each v

Depth-First Search

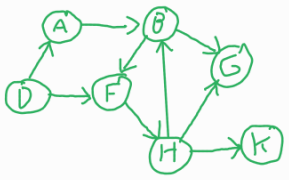
```
DFS (G)
1 for each vertex u ∈ V[G]
2     do colour[u] ← WHITE   - undiscovered
3        π[u] ← NIL
4 time ← 0
5 for each vertex u ∈ V[G]
6     do if colour[u] = WHITE
7        then DFS-VISIT(u)

DFS-VISIT(u)
1 colour[u] ← GREY   - discovered, unprocessed
2 time ← time+1
3 d[u] ← time
4 for each vertex v ∈ Adj[u]
5     do if colour[v] = WHITE
6        then π[v] ← u
7             DFS-VISIT(v)
8 colour[u] ← BLACK   - discovered, processed
9 f[u] ← time ← time+1
```

Process:

- initialise all nodes as white and set initial time to 0.
- for each white node in the graph:
  - set its colour to grey
  - increment time
  - set its discovered time
  - for each white neighbour
    - set their predecessor
    - recursively call on the neighbour
  - set its colour to black
  - set its finish time

Time complexity: $O(V+E)$

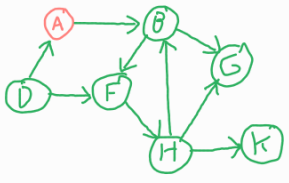
Example



time = 0

	A	B	D	F	G	H	K
Colour	•	•	•	•	•	•	•
discovery time, d							
finish time, f							
Predecessor, π	NULL	NULL	NULL	NULL	NULL	NULL	NULL

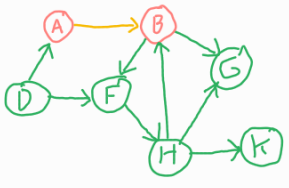
↓ DFS-VISIT (A)



time = 1

	A	B	D	F	G	H	K
Colour	•	•	•	•	•	•	•
discovery time, d	1						
finish time, f							
Predecessor, π	NULL	NULL	NULL	NULL	NULL	NULL	NULL

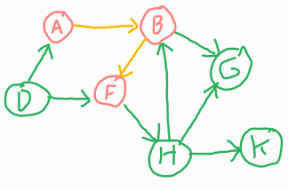
↓ DFS-VISIT (B)



time = 2

	A	B	D	F	G	H	K
Colour	•	•	•	•	•	•	•
discovery time, d	1	2					
finish time, f							
Predecessor, π	NULL	A	NULL	NULL	NULL	NULL	NULL

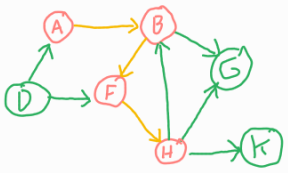
↓ DFS-VISIT (F)



time = 3

	A	B	D	F	G	H	K
Colour	•	•	•	•	•	•	•
discovery time, d	1	2		3			
finish time, f							
Predecessor, π	NULL	A	NULL	B	NULL	NULL	NULL

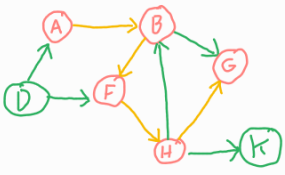
↓ DFS-VISIT (H)



time = 4

	A	B	D	F	G	H	K
Colour	•	•	•	•	•	•	•
discovery time, d	1	2		3		4	
finish time, F							
Predecessor, π	NULL	A	NULL	B	NULL	F	NULL

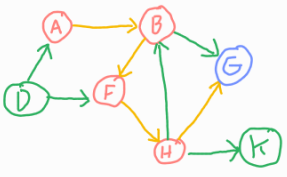
↓ DFS-VISIT(G)



time = 5

	A	B	D	F	G	H	K
Colour	•	•	•	•	•	•	•
discovery time, d	1	2		3	5	4	
finish time, F							
Predecessor, π	NULL	A	NULL	B	H	F	NULL

↓ No white neighbours of G
Backtrack

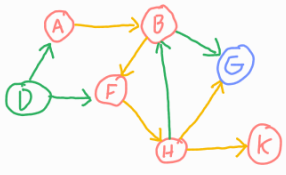


time = 6

	A	B	D	F	G	H	K
Colour	•	•	•	•	•	•	•
discovery time, d	1	2		3	5	4	
finish time, F					6		
Predecessor, π	NULL	A	NULL	B	H	F	NULL

↓ DFS-VISIT(K)

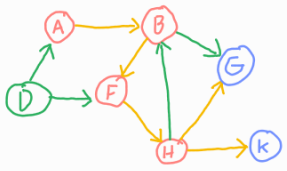
backtrack



time = 7

	A	B	D	F	G	H	K
Colour	•	•	•	•	•	•	•
discovery time, d	1	2		3	5	4	7
finish time, F					6		
Predecessor, π	NULL	A	NULL	B	H	F	F

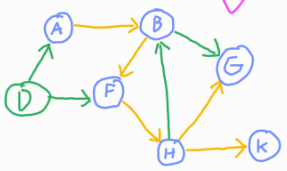
backtrack



time = 8

	A	B	D	F	G	H	K
Colour	•	•	•	•	•	•	•
discovery time, d	1	2		3	5	4	7
finish time, F					6		8
Predecessor, π	NULL	A	NULL	B	H	F	F

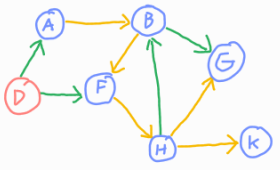
↓ backtrack x4



time = 12

	A	B	D	F	G	H	K
Colour	•	•	•	•	•	•	•
discovery time, d	1	2		3	5	4	7
finish time, F	12	11		10	6	9	8
Predecessor, π	NULL	A	NULL	B	H	F	F

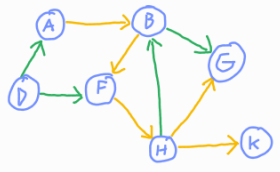
↓ DFS-VISIT(D)



time = 13

	A	B	D	F	G	H	K
Colour	•	•	•	•	•	•	•
discovery time, d	1	2	13	3	5	4	7
finish time, F	12	11		10	6	9	8
Predecessor, π	NULL	A	NULL	B	H	F	F

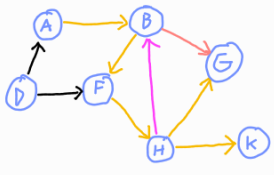
↓



time = 14

	A	B	D	F	G	H	K
Colour	•	•	•	•	•	•	•
discovery time, d	1	2	13	3	5	4	7
finish time, F	12	11	14	10	6	9	8
Predecessor, π	NULL	A	NULL	B	H	F	F

The highlighted edges form the depth first forest of G.
From this, the edges of G can be classified:



- Tree edges are edges in the DFS-forest
- Forward edges are those NOT in the forest which connect a node to one of its descendents
- back edges are those not in the forest connecting a node to its ancestor
- Cross edges are those between nodes that are in separate trees within the forest.

G is a descendent of B in the DFS-Forest so (B,G) is a forward edge ←

Note: in undirected graphs, forward and back edges are the same thing so we call them back edges. Undirected graphs have only tree edges and back edges.

Minimum Spanning Trees

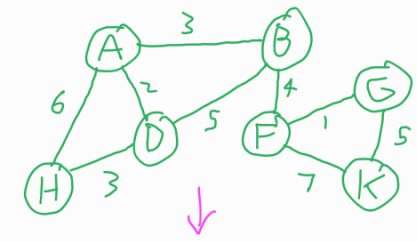
An MST of a weighted graph is a subset of the edges of the graph that connects all the vertices with the least total weight.

MSTs:
- have $|V|-1$ edges
- have no cycles
- may or may not be unique

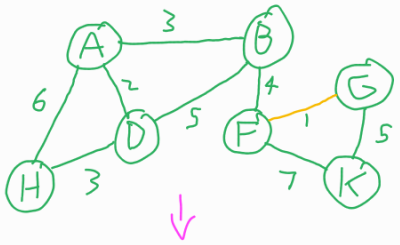
Note: weighted graphs are represented using an adjacency matrix, with ∞ used if no edge exists.

Kruskal's Algorithm:
- Sort edges by weight
- Initialise $A = \emptyset$
- In increasing order of weight, add each edge e to A unless a cycle is created.
- Time complexity: $O(E \log V)$

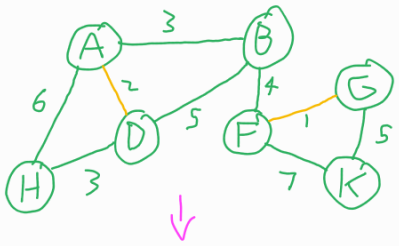
Example



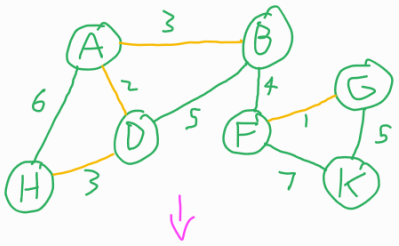
$A = \emptyset$



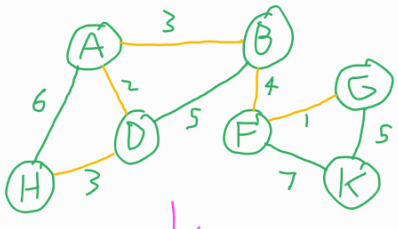
$A = FG$



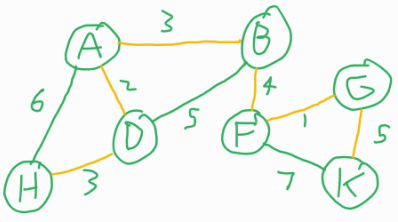
$A = FG, AD$



$A = FG, AD, AB, DH$



$A = FG, AD, AB, DH, BF$



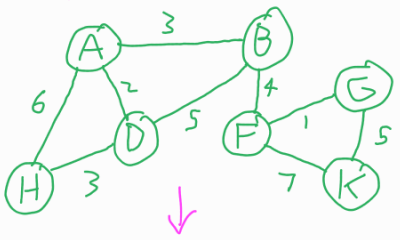
$A = FG, AD, AB, DH, BF$

Prim's Algorithm:

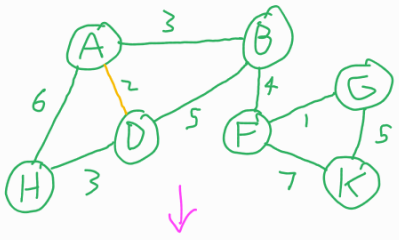
- Let $U = \{u\}$, where u is an arbitrary vertex in G
- Let $A = \emptyset$
- Until U contains all vertices, find the least-weight edge that joins a vertex in U to one not in U and add it to A and the other vertex to U.
- Time Complexity $O(V \log V + E)$

Both Kruskal's and Prim's algorithms are special cases of a generic greedy MST algorithm which iteratively builds up a set A which is a subset of some MST. To find the next edge to add, cut the graph wrt A (i.e. partition the graph such that all edges in A are in one partition) and add the edge of smallest weight which crosses the cut.

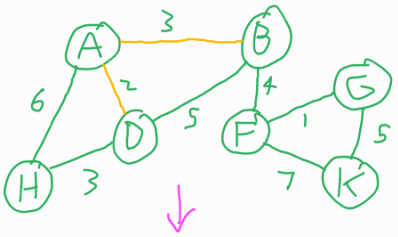
Example



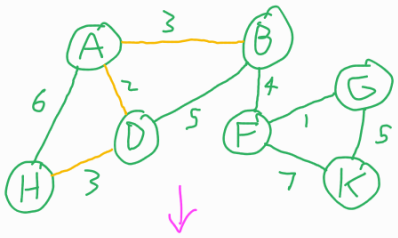
$A = \emptyset$
$U = A$



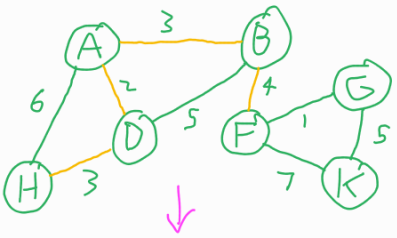
$A = AD$
$U = A, D$



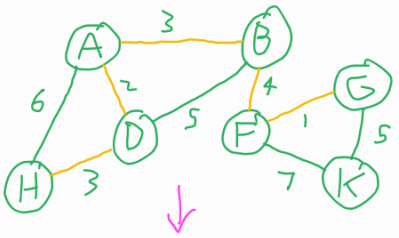
$A = AD, AB$
$U = A, D, B$



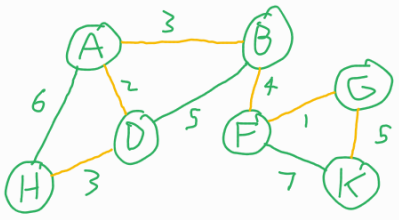
$A = AD, AB, DH$
$U = A, D, B, H$



$A = AD, AB, DH, BF$
$U = A, D, B, H, F$



$A = AD, AB, DH, BF, FG$
$U = A, D, B, H, F, G$



$A = AD, AB, DH, BF, FG, GK$
$U = A, D, B, H, F, G, K$